

Pools: Fonds as Passive Data Resources

What Is a Fond? I

A **fond** is a versioned, read-only data pool that any project in the repository can consume without modifying. The term evokes the culinary concept of a concentrated stock — a stable base that enriches whatever is built on top of it. Fonds live under the top-level `fonds/<scope>/<name>/` directory, each containing a manifest file (`fonds.yaml`), a `data/` subdirectory, and optional documentation. This architecture separates *data ownership* from *data usage*: research projects in `projects/` are consumers, not producers, of fond data. The separation prevents the accretion of project-specific mutations in shared resources — a recurring source of reproducibility failures in collaborative research software [Wilson2014best].

What Is a Fond? II

The three-layer taxonomy (bibliography, contacts, datasets) maps to the three most common categories of curated research data: citable literature, human collaboration networks, and input/output datasets. Each category carries its own schema enforced by `rules/templates/template_manuscript_rules`.

The Three Template Fonds I

The Three Template Fonds II

template_bibliography

The `template_bibliography` fond is a curated reference library stored in two formats: a BibTeX file (`data/references.bib`) as source of truth, and a flat CSV export (`data/references.csv`) for programmatic querying. Deduplication is enforced on the cite key (the primary CSV column). The collection spans foundational machine-learning works — the transformer architecture [`@Vaswani2017attention`], early convolutional network research [`@LeCun1998gradient`], and large-scale language model pre-training [`@Brown2020gpt3`] — alongside software-engineering references on best practices [`@Wilson2014best`] and robust software design [`@Taschuk2017ten`]. In the current integration run, the fond contains **`{{integration.bib_entries}}` entries**.

The bibliography fond illustrates the *registry pattern* [`@Fowler2002patterns`]: a single authoritative list is maintained centrally, and all projects reference it rather than keeping private copies. This guarantees citation consistency across all exemplar manuscripts.

The fonds.yaml Manifest I

Every fond root must contain a fonds.yaml manifest with at minimum three fields:

```
type: bibliography    # bibliography / contacts / datasets
description: "Human-readable description of the fond"
version: "1.0"
tags: [curated, exemplar]
```

The type field governs which reader function is appropriate and what schema the data/ directory is expected to follow. The version field is incremented whenever the schema changes, enabling consumers to detect and handle schema drift without silent failures.

The `fonds_reader` Module I

The `src/fonds_reader.py` module provides three reader functions — one per fond type — plus a convenience aggregator:

```
from src.fonds_reader import (  
    read_bibliography_fond,  
    read_contacts_fond,  
    read_datasets_fond,  
    read_all_fonds,  
)
```

```
bib          = read_bibliography_fond()    # dict / None  
contacts    = read_contacts_fond()        # dict / None  
datasets    = read_datasets_fond()        # dict / None  
all_fonds   = read_all_fonds()            # {"bibliography": .
```

The `fonds_reader` Module II

Each reader resolves the repository root from `pathlib.Path(__file__).resolve().parents[4]` and uses a `try/except` block that catches both `FileNotFoundError` and `yaml.YAMLError`. Returning `None` on failure rather than raising ensures that the integration pipeline degrades gracefully when a fond has not yet been populated by a parallel agent [Taschuk2017]. In the current run, `{integration.fonds_loaded}` of 3 expected fonds were successfully loaded (see @fig:counts).

Resilience by Design I

The fond layer enforces resilience at two levels. At the **structural** level, readers tolerate missing fonds entirely. At the **schema** level, the manifest version field allows consumers to check compatibility before processing data. This two-level approach means a fond can evolve its schema without breaking existing consumers that have not yet been updated: the consumer detects the version mismatch and either adapts or skips, rather than crashing silently on malformed data.